```
!pip install prophet yfinance holidays -q
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import yfinance as yf
import holidays
import warnings
warnings.filterwarnings('ignore')

from prophet import Prophet
from prophet.diagnostics import cross_validation, performance_metrics
from prophet.plot import plot_cross_validation_metric
```

```
from datetime import date

today     = '2026-02-27'
start     = '2010-01-01'

print(f"Downloading NIFTY data from {start} to {today}...")

df = yf.download("^NSEI", start=start, end=today)
df.columns = df.columns.get_level_values(0)
df.reset_index(inplace=True)

print(f"Downloaded {len(df)} rows")
print(f"Range: {df['Date'].min().date()} → {df['Date'].max().date()}")
print(f"Last close: {df['Close'].iloc[-1]:.2f}")
print(df.tail())
```

```
Downloading NIFTY data from 2010-01-01 to 2026-02-27...
[*********************100%***********************]  1 of 1 completedDownloaded 3968 rows
Range: 2010-01-04 → 2026-02-26
Last close: 25496.55
Price       Date         Close          High           Low          Open  \
3963  2026-02-20  25571.250000  25663.550781  25379.750000  25406.550781
3964  2026-02-23  25713.000000  25771.449219  25609.349609  25678.400391
3965  2026-02-24  25424.650391  25641.800781  25327.599609  25641.800781
3966  2026-02-25  25482.500000  25652.599609  25428.199219  25512.599609
3967  2026-02-26  25496.550781  25572.949219  25400.949219  25556.300781

Price  Volume
3963   296600
3964   313700
3965   439100
3966   414200
3967   405200
```

```
prophet_df = df[['Date', 'Close']].copy()
prophet_df.columns = ['ds', 'y']
prophet_df['ds'] = pd.to_datetime(prophet_df['ds'])
prophet_df.dropna(inplace=True)

print(f"Prophet data ready!")
print(f"From: {prophet_df['ds'].iloc[0].date()}")
print(f"To:   {prophet_df['ds'].iloc[-1].date()}")
print(f"Last close: {prophet_df['y'].iloc[-1]:.2f}")
print(f"Total rows: {len(prophet_df)}")
```

```
Prophet data ready!
From: 2010-01-04
To:   2026-02-26
Last close: 25496.55
Total rows: 3968
```

```
current_year = date.today().year

india_hols = holidays.India(years=range(2010, current_year + 2))

indian_holidays_df = pd.DataFrame({
    'holiday'      : 'indian_market_holiday',
    'ds'           : pd.to_datetime(list(india_hols.keys())),
    'lower_window' : -1,
    'upper_window' : 1,
}).sort_values('ds').reset_index(drop=True)

print(f"Total holidays loaded: {len(indian_holidays_df)}")
print(indian_holidays_df.head(10))
```

```
Total holidays loaded: 285
                 holiday         ds  lower_window  upper_window
0  indian_market_holiday 2010-01-26            -1             1
1  indian_market_holiday 2010-02-12            -1             1
2  indian_market_holiday 2010-02-27            -1             1
3  indian_market_holiday 2010-04-02            -1             1
4  indian_market_holiday 2010-04-28            -1             1
5  indian_market_holiday 2010-05-27            -1             1
6  indian_market_holiday 2010-08-15            -1             1
7  indian_market_holiday 2010-09-02            -1             1
8  indian_market_holiday 2010-09-10            -1             1
9  indian_market_holiday 2010-10-02            -1             1
```

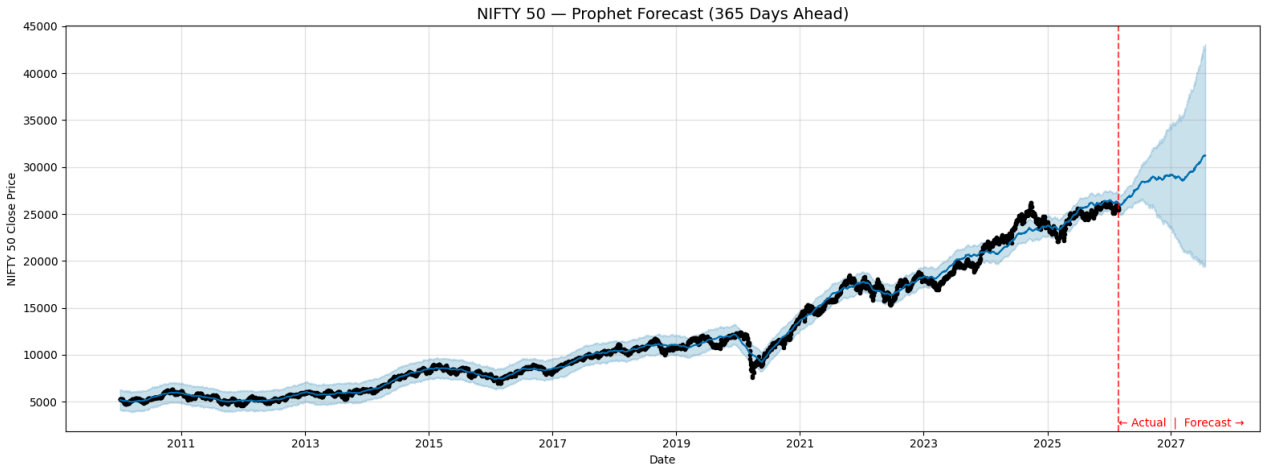
```
model = Prophet(
    changepoint_prior_scale = 0.15,
    n_changepoints          = 30,
    yearly_seasonality      = True,
    weekly_seasonality      = True,
    daily_seasonality       = True,
    seasonality_prior_scale = 10,
    seasonality_mode        = 'multiplicative',
    holidays                = indian_holidays_df,
    holidays_prior_scale    = 10,
    interval_width          = 0.95,
)

print("⏳ Training model...")
model.fit(prophet_df)
print(f"Model trained on data up to: {prophet_df['ds'].iloc[-1].date()}")
print(f"Last known NIFTY close: {prophet_df['y'].iloc[-1]:.2f}")
```

```
⏳ Training model...
Model trained on data up to: 2026-02-26
Last known NIFTY close: 25496.55
```

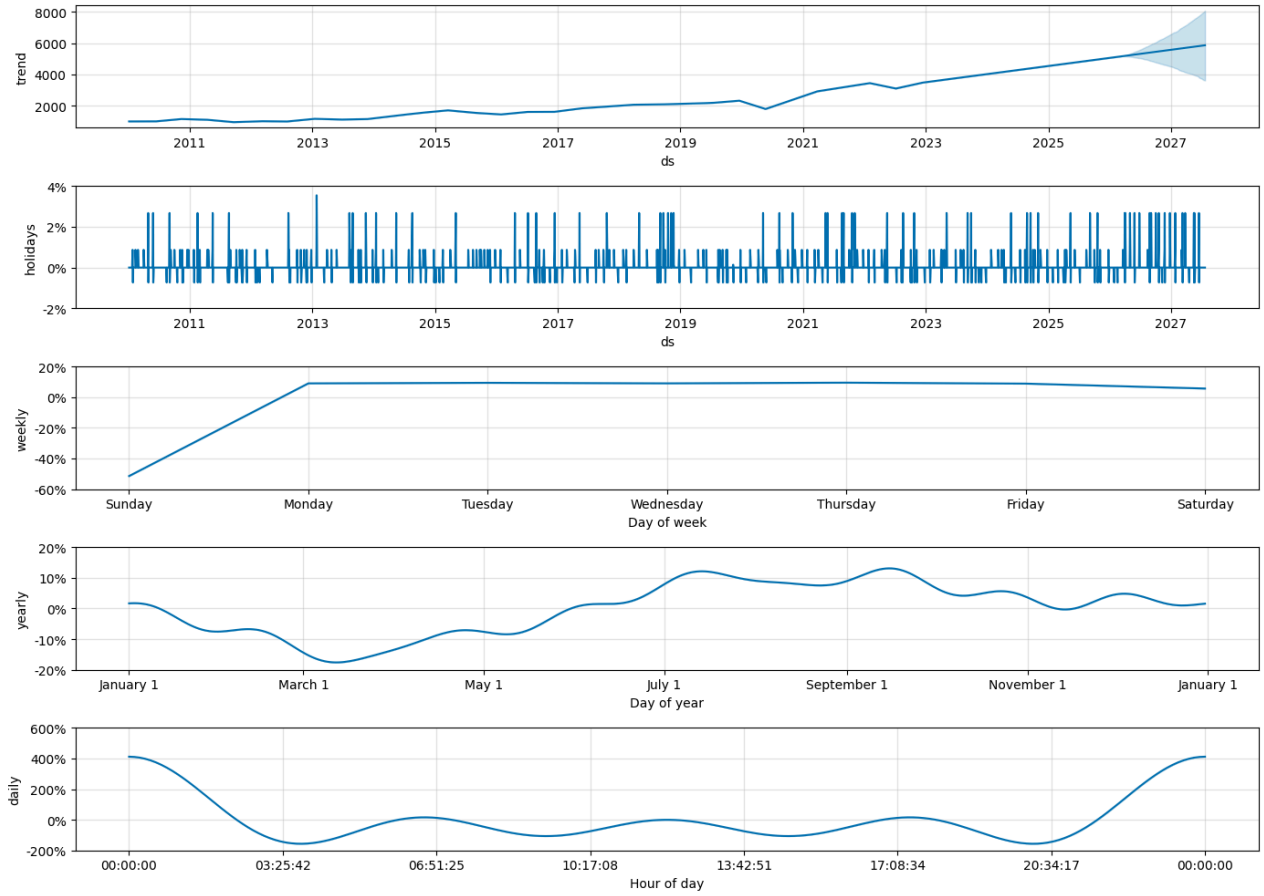
```
future   = model.make_future_dataframe(periods=365, freq='B')
forecast = model.predict(future)

print(f"Forecast generated!")
print(f"From: {forecast['ds'].min().date()}")
print(f"To:   {forecast['ds'].max().date()}")
```

```
Forecast generated!
From: 2010-01-04
To:   2027-07-22
```

```
fig, ax = plt.subplots(figsize=(16, 6))

model.plot(forecast, ax=ax)

ax.set_title('NIFTY 50 — Prophet Forecast (365 Days Ahead)', fontsize=14)
ax.set_xlabel('Date')
ax.set_ylabel('NIFTY 50 Close Price')

last_date = prophet_df['ds'].max()
ax.axvline(x=last_date, color='red', linestyle='--', alpha=0.7)
ax.text(last_date, ax.get_ylim()[0] + 500,
        '← Actual  |  Forecast →',
        color='red', fontsize=10)

plt.tight_layout()
plt.show()
```




```
fig2 = model.plot_components(forecast, figsize=(14, 10))
plt.tight_layout()
plt.show()
```




```
print("Running cross validation (takes 2-3 mins)...")

cv_results = cross_validation(
    model,
    initial  = '1825 days',
    period   = '180 days',
    horizon  = '365 days',
    parallel = "processes"
)

print("Cross validation done!")
print(cv_results[['ds', 'y', 'yhat', 'yhat_lower', 'yhat_upper']].head(10))
```

```
INFO:prophet:Making 21 forecasts with cutoffs between 2015-04-20 00:00:00 and 2025-02-26 00:00:00
INFO:prophet:Applying in parallel with <concurrent.futures.process.ProcessPoolExecutor object at 0x78037c3948f0>
Running cross validation (takes 2-3 mins)...
```

```
Cross validation done!
          ds            y         yhat   yhat_lower   yhat_upper
0 2015-04-21  8377.750000  8926.403884  8668.368664  9167.404630
1 2015-04-22  8429.700195  8943.387347  8713.066611  9182.730876
2 2015-04-23  8398.299805  8957.467881  8713.141855  9198.794024
3 2015-04-24  8305.250000  8960.567919  8684.547875  9216.477211
4 2015-04-27  8213.799805  8985.080915  8744.105846  9245.578493
5 2015-04-28  8285.599609  8991.387087  8750.675398  9246.718618
6 2015-04-29  8239.750000  9007.071654  8754.017772  9268.460758
7 2015-04-30  8181.500000  9020.316292  8744.006640  9282.682112
8 2015-05-04  8331.950195  9013.795890  8763.202109  9257.958350
9 2015-05-05  8324.799805  9099.027536  8837.666252  9359.362537
```

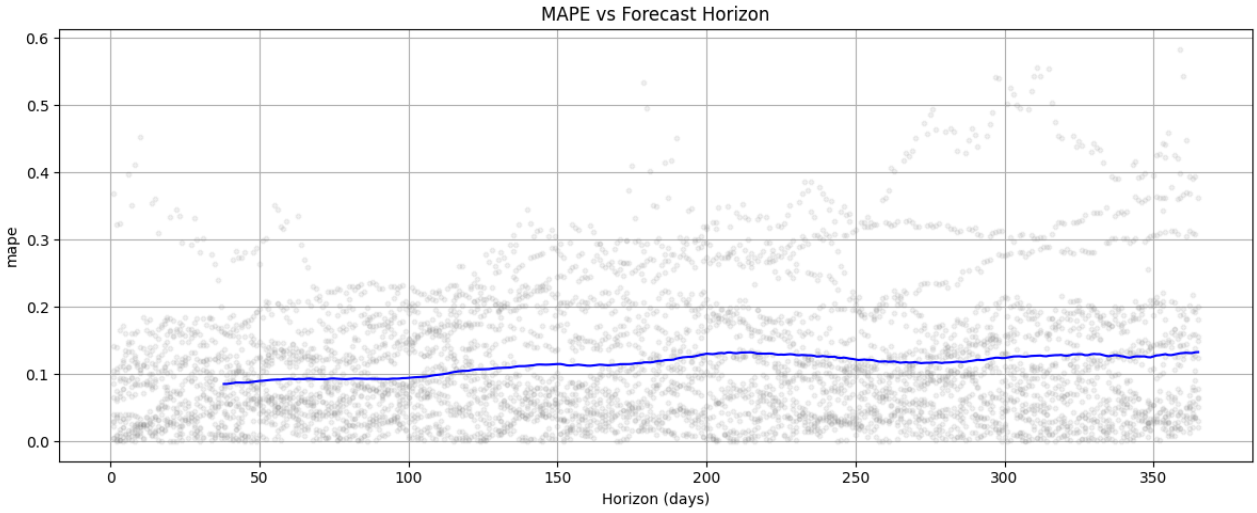
```
metrics = performance_metrics(cv_results)

print("Model Performance:")
print(metrics[['horizon', 'mae', 'rmse', 'mape']].to_string())

fig3 = plot_cross_validation_metric(cv_results, metric='mape', figsize=(12, 5))
plt.title('MAPE vs Forecast Horizon')
plt.tight_layout()
plt.show()
```

```
Model Performance:
     horizon          mae         rmse      mape
0    38 days  1211.813148  1605.976019  0.085634
1    39 days  1212.400177  1604.234472  0.085652
2    40 days  1226.588494  1616.587525  0.086202
3    41 days  1237.515074  1626.602968  0.086891
4    42 days  1243.190077  1630.969048  0.087670
5    43 days  1236.400263  1623.318111  0.087704
6    44 days  1233.191372  1619.523601  0.087559
7    45 days  1231.201577  1617.094750  0.087544
8    46 days  1232.728965  1613.686651  0.087809
9    47 days  1244.211698  1621.195496  0.088417
10   48 days  1252.445041  1625.520911  0.088958
11   49 days  1254.176745  1624.458062  0.089640
12   50 days  1251.953515  1618.818508  0.090107
13   51 days  1249.823641  1613.884085  0.090371
14   52 days  1255.309423  1615.892625  0.090868
15   53 days  1261.995333  1618.283736  0.091313
16   54 days  1269.946009  1621.881092  0.091734
17   55 days  1267.977400  1618.104037  0.091935
18   56 days  1262.380735  1610.492113  0.092221
19   57 days  1255.000246  1599.852739  0.092122
20   58 days  1258.652868  1600.409348  0.092737
21   59 days  1260.602539  1600.225979  0.093058
22   60 days  1264.478783  1599.888424  0.093219
23   61 days  1265.814424  1599.166318  0.093058
24   62 days  1258.050612  1591.104718  0.092601
25   63 days  1256.445973  1585.632218  0.093049
26   64 days  1250.416178  1575.773461  0.092919
27   65 days  1248.135795  1569.584098  0.092888
28   66 days  1256.519641  1573.993111  0.093443
29   67 days  1258.739414  1571.643078  0.093547
30   68 days  1252.982275  1565.678383  0.093042
31   69 days  1246.656767  1558.205679  0.092773
32   70 days  1240.223786  1548.293887  0.092711
33   71 days  1235.288163  1541.255517  0.092515
34   72 days  1236.720486  1540.416280  0.092624
35   73 days  1241.677977  1543.701184  0.093049
36   74 days  1247.797841  1546.732404  0.093655
37   75 days  1249.490384  1548.946931  0.093736
38   76 days  1237.441209  1536.893000  0.093332
39   77 days  1228.566766  1525.954039  0.093114
40   78 days  1220.098415  1515.133606  0.092752
41   79 days  1221.158264  1515.152059  0.092726
42   80 days  1226.191900  1521.667506  0.093091
43   81 days  1232.670298  1526.948866  0.093413
44   82 days  1239.026319  1536.666054  0.093694
45   83 days  1231.588226  1531.500580  0.093531
46   84 days  1225.549224  1525.605056  0.093445
47   85 days  1222.925729  1523.419366  0.093238
48   86 days  1222.192920  1524.174002  0.093133
49   87 days  1228.173249  1532.037234  0.093330
50   88 days  1228.633197  1532.863300  0.093254
51   89 days  1227.251370  1534.258790  0.093032
52   90 days  1222.391412  1530.590266  0.092992
53   91 days  1221.725295  1528.406395  0.092927
54   92 days  1223.755640  1528.843546  0.092888
55   93 days  1223.689714  1525.571893  0.092899
56   94 days  1229.957162  1531.817323  0.093229
57   95 days  1237.925309  1540.920818  0.093601
58   96 days  1236.689342  1542.443876  0.093497
59   97 days  1235.753999  1540.200923  0.093731
60   98 days  1241.304255  1543.716590  0.094280
61   99 days  1244.110477  1545.520341  0.094356
62  100 days  1250.489806  1552.465438  0.094674
63  101 days  1262.308173  1566.886835  0.095177
64  102 days  1268.449569  1576.947671  0.095560
65  103 days  1272.032175  1585.765090  0.095743
66  104 days  1272.615240  1586.025608  0.095967
67  105 days  1276.335050  1587.112715  0.096278
68  106 days  1279.614559  1588.257862  0.096610
69  107 days  1292.051804  1602.577997  0.097283
70  108 days  1305.784167  1618.079994  0.098159
71  109 days  1314.606176  1629.886611  0.098809
72  110 days  1321.551226  1640.681481  0.099250
73  111 days  1325.439735  1642.759393  0.099798
74  112 days  1334.144786  1647.586712  0.100475
75  113 days  1347.730053  1660.803164  0.101199
76  114 days  1363.964454  1678.376333  0.102175
77  115 days  1379.140920  1694.219839  0.103187
78  116 days  1389.084401  1707.430261  0.103904
79  117 days  1398.134018  1720.514986  0.104525
80  118 days  1401.792611  1723.825845  0.104910
81  119 days  1404.322753  1725.088515  0.105035
82  120 days  1410.603313  1733.472314  0.105214
83  121 days  1423.981515  1748.953392  0.106002
84  122 days  1436.764492  1765.178523  0.106866
85  123 days  1443.843534  1778.814598  0.107284
86  124 days  1444.381522  1786.611399  0.107300
87  125 days  1445.781996  1789.530878  0.107494
```

```
88  126 days  1445.252504  1790.639257  0.107583
89  127 days  1447.880008  1797.040172  0.107742
90  128 days  1457.260511  1810.159176  0.108340
91  129 days  1466.611536  1823.013421  0.108968
92  130 days  1470.839058  1831.461286  0.109347
93  131 days  1472.346909  1837.989104  0.109438
94  132 days  1473.938504  1840.602690  0.109697
95  133 days  1474.530679  1840.786587  0.109851
96  134 days  1477.458924  1847.002646  0.110023
97  135 days  1488.403581  1861.073471  0.110811
98  136 days  1494.119438  1867.566643  0.111449
99  137 days  1499.567494  1879.042255  0.112055
100 138 days  1501.711869  1885.626292  0.112044
101 139 days  1499.879493  1884.260470  0.112141
102 140 days  1503.012492  1885.984482  0.112590
103 141 days  1509.372104  1897.207492  0.112979
104 142 days  1521.827870  1915.396952  0.113450
105 143 days  1530.964577  1929.083888  0.114323
106 144 days  1533.960943  1938.474354  0.114553
107 145 days  1535.693197  1947.545891  0.114384
108 146 days  1531.297608  1942.107653  0.114250
109 147 days  1533.291190  1943.719938  0.114372
110 148 days  1539.443341  1955.713046  0.114683
111 149 days  1546.693209  1967.728924  0.115087
112 150 days  1553.437923  1981.235510  0.115215
113 151 days  1554.337186  1989.040637  0.115249
114 152 days  1551.516541  1992.898271  0.114504
115 153 days  1536.148334  1977.164381  0.113382
116 154 days  1531.553953  1973.962634  0.113071
117 155 days  1539.628365  1982.203711  0.113333
118 156 days  1548.010767  1990.496829  0.113914
119 157 days  1552.866446  1998.515825  0.114269
120 158 days  1549.328039  1999.300513  0.113836
121 159 days  1546.833879  1999.731208  0.113288
122 160 days  1540.557526  1991.416019  0.112742
123 161 days  1539.215891  1987.719751  0.112703
124 162 days  1547.860670  1996.014452  0.113103
125 163 days  1558.641758  2006.791780  0.113618
126 164 days  1570.855206  2022.856943  0.114274
127 165 days  1569.995574  2027.155026  0.114096
128 166 days  1570.557371  2034.163440  0.113620
129 167 days  1569.369986  2032.178148  0.113373
130 168 days  1571.723451  2031.790467  0.113583
131 169 days  1580.367670  2041.324206  0.113934
132 170 days  1588.979261  2055.327833  0.114314
133 171 days  1604.160572  2075.092949  0.115018
134 172 days  1605.336359  2079.128721  0.115037
135 173 days  1609.358208  2087.950246  0.115020
136 174 days  1607.758722  2083.448617  0.115119
137 175 days  1606.963782  2077.728155  0.115425
138 176 days  1620.052682  2090.417694  0.116271
139 177 days  1629.102564  2101.827293  0.116485
140 178 days  1635.437808  2111.282999  0.116707
141 179 days  1644.940709  2123.895821  0.117684
142 180 days  1650.223734  2134.969384  0.117806
143 181 days  1649.213604  2133.485598  0.117602
144 182 days  1649.407436  2127.887345  0.118068
145 183 days  1664.143223  2142.152612  0.119126
146 184 days  1680.491479  2160.029882  0.119879
147 185 days  1690.522702  2173.843335  0.119967
148 186 days  1701.697422  2187.876333  0.120914
149 187 days  1704.990280  2191.116477  0.121359
150 188 days  1703.693057  2187.086615  0.121396
151 189 days  1704.495876  2181.331346  0.121794
152 190 days  1721.665646  2193.333278  0.123410
153 191 days  1740.225449  2209.208330  0.124342
154 192 days  1756.543391  2229.917907  0.125203
155 193 days  1764.516718  2245.016111  0.125607
156 194 days  1770.913215  2252.193354  0.125913
157 195 days  1769.698253  2249.082467  0.125970
158 196 days  1772.714935  2246.759589  0.126756
159 197 days  1786.644203  2257.758465  0.127725
160 198 days  1808.093527  2278.352656  0.128892
161 199 days  1824.111089  2300.391233  0.129616
162 200 days  1838.831729  2320.535939  0.130595
163 201 days  1838.614381  2324.434397  0.130293
164 202 days  1831.703761  2318.508884  0.129943
165 203 days  1831.657704  2314.920331  0.130422
166 204 days  1840.781191  2320.877488  0.131360
167 205 days  1851.437332  2332.255377  0.131635
168 206 days  1851.312526  2340.863249  0.131185
169 207 days  1866.997587  2360.888129  0.132346
170 208 days  1870.895045  2367.620244  0.132470
171 209 days  1856.445146  2354.905830  0.131624
172 210 days  1848.421904  2343.952920  0.131356
173 211 days  1852.973426  2345.164883  0.131830
174 212 days  1866.535666  2357.979325  0.132370
175 213 days  1872.554522  2371.112559  0.132402
176 214 days  1879.054108  2384.758105  0.132582
177 215 days  1881.132846  2389.087412  0.132655
178 216 days  1869.024034  2377.448712  0.131956
```

179	217 days	1855.245298	2360.903080	0.130810
180	218 days	1851.727857	2351.280900	0.130738
181	219 days	1851.555129	2350.439399	0.130508
182	220 days	1861.045728	2364.303051	0.130602
183	221 days	1865.511768	2372.409540	0.130871
184	222 days	1861.137835	2368.745101	0.130691
185	223 days	1844.632833	2349.427889	0.129723
186	224 days	1832.879542	2333.151707	0.129098
187	225 days	1832.859732	2329.751177	0.129309
188	226 days	1830.829577	2327.856667	0.129060
189	227 days	1841.850745	2341.611017	0.129513
190	228 days	1846.713718	2349.470446	0.129859
191	229 days	1840.269328	2344.482678	0.129541
192	230 days	1823.568001	2325.597475	0.128582
193	231 days	1811.622396	2307.474647	0.128292
194	232 days	1813.930237	2305.897853	0.128727
195	233 days	1811.658314	2306.082480	0.128389
196	234 days	1810.894940	2309.244407	0.128122
197	235 days	1803.988197	2303.015402	0.128097
198	236 days	1793.198770	2290.831557	0.127558
199	237 days	1774.922065	2268.155719	0.126394
200	238 days	1767.860262	2255.780986	0.126530
201	239 days	1768.936446	2252.789721	0.127118
202	240 days	1765.951985	2250.019617	0.126563
203	241 days	1763.374976	2250.772407	0.125966
204	242 days	1758.869775	2246.908063	0.125961
205	243 days	1757.498980	2238.346380	0.126176
206	244 days	1735.131876	2210.604341	0.124648
207	245 days	1721.703600	2189.839814	0.124192
208	246 days	1730.044931	2193.739312	0.124845
209	247 days	1726.588686	2192.078458	0.124220
210	248 days	1716.593220	2184.117957	0.123295
211	249 days	1711.097483	2175.827298	0.122832
212	250 days	1700.849974	2160.361544	0.122113
213	251 days	1686.237023	2139.499528	0.121220
214	252 days	1685.295462	2132.636766	0.121497
215	253 days	1690.777764	2135.146730	0.121947
216	254 days	1684.632099	2130.325350	0.121385
217	255 days	1684.121103	2129.629491	0.121444
218	256 days	1673.993203	2115.251174	0.120855
219	257 days	1659.055191	2098.136021	0.119805
220	258 days	1642.768573	2077.668690	0.118811
221	259 days	1640.589812	2070.836464	0.118932
222	260 days	1644.699477	2075.921117	0.119242
223	261 days	1637.967141	2072.721336	0.118473
224	262 days	1641.099368	2076.992791	0.118672
225	263 days	1639.956191	2072.545169	0.119057
226	264 days	1627.630708	2059.080134	0.118044
227	265 days	1615.716722	2042.236008	0.117197
228	266 days	1617.334549	2038.398965	0.117645
229	267 days	1625.087764	2049.210195	0.118039
230	268 days	1618.555161	2046.141647	0.117319
231	269 days	1624.148994	2050.185462	0.117622
232	270 days	1625.243711	2048.125672	0.118130
233	271 days	1617.922405	2040.672496	0.117220
234	272 days	1609.064567	2026.845397	0.116518
235	273 days	1608.601302	2021.572611	0.116923
236	274 days	1612.764934	2023.700804	0.117141
237	275 days	1616.562308	2027.211638	0.117044
238	276 days	1622.013257	2032.755522	0.117523
239	277 days	1619.786056	2029.425287	0.117851
240	278 days	1613.886606	2022.316427	0.117347
241	279 days	1605.601076	2009.231771	0.116710
242	280 days	1613.216864	2013.494852	0.117410
243	281 days	1622.956073	2023.623646	0.117610
244	282 days	1626.919723	2029.197640	0.117727
245	283 days	1634.931330	2037.056830	0.118314
246	284 days	1634.951728	2036.749056	0.118749
247	285 days	1631.497286	2031.964639	0.118487
248	286 days	1624.775132	2022.667095	0.118145
249	287 days	1632.145961	2030.248480	0.118598
250	288 days	1643.891142	2042.724482	0.119431
251	289 days	1651.138499	2049.640980	0.119902
252	290 days	1660.523866	2059.344171	0.120601
253	291 days	1665.679786	2064.882193	0.121425
254	292 days	1662.680049	2063.719665	0.121206
255	293 days	1663.452166	2062.412150	0.121177
256	294 days	1676.062213	2074.186524	0.122208
257	295 days	1687.717036	2085.254698	0.123134
258	296 days	1696.565622	2093.525307	0.123774
259	297 days	1706.204239	2105.696571	0.124594
260	298 days	1704.267998	2108.134304	0.124624
261	299 days	1698.032792	2104.424037	0.124062
262	300 days	1700.265296	2107.179926	0.124087
263	301 days	1710.363007	2119.100972	0.124851
264	302 days	1720.368472	2130.067648	0.125781
265	303 days	1729.749724	2138.616574	0.126467
266	304 days	1730.766406	2140.219910	0.126616
267	305 days	1730.824113	2142.950664	0.126784
268	306 days	1725.801178	2139.945424	0.126276

```
269 307 days  1723.634614  2139.760395  0.126059
270 308 days  1733.614080  2152.902625  0.126722
271 309 days  1740.836154  2162.951159  0.127245
272 310 days  1742.964042  2167.551564  0.127401
273 311 days  1744.789731  2171.101429  0.127635
274 312 days  1743.165757  2170.496460  0.127832
275 313 days  1733.374683  2162.386621  0.127144
276 314 days  1729.479585  2159.718491  0.126818
277 315 days  1738.317249  2171.400377  0.127634
278 316 days  1741.031663  2178.367598  0.128061
279 317 days  1743.421391  2185.510260  0.128387
280 318 days  1744.406801  2191.248896  0.128459
281 319 days  1742.072364  2192.427036  0.128756
282 320 days  1734.149524  2187.115495  0.127887
283 321 days  1730.064921  2183.285116  0.127620
284 322 days  1743.885781  2197.909534  0.128348
285 323 days  1757.432417  2214.148903  0.129634
286 324 days  1759.273768  2219.443705  0.129557
287 325 days  1762.495733  2227.524362  0.129795
288 326 days  1754.979607  2224.143851  0.129736
289 327 days  1742.186501  2216.895647  0.128426
290 328 days  1744.849463  2217.366981  0.128444
291 329 days  1758.074389  2230.261464  0.129660
292 330 days  1767.048669  2245.197020  0.130251
293 331 days  1767.146371  2251.517552  0.129841
294 332 days  1770.541808  2258.646784  0.129870
295 333 days  1752.172852  2241.003597  0.128631
296 334 days  1741.318868  2235.018062  0.127511
297 335 days  1739.952143  2230.316617  0.126908
298 336 days  1746.610641  2234.187774  0.127664
299 337 days  1754.725846  2244.501804  0.128322
300 338 days  1756.177616  2246.374428  0.127924
301 339 days  1752.031429  2245.872872  0.127064
302 340 days  1745.636902  2243.540748  0.126466
303 341 days  1733.955321  2236.461901  0.125222
304 342 days  1728.945266  2227.823544  0.124703
305 343 days  1737.851808  2232.952128  0.125746
306 344 days  1749.633654  2248.574061  0.126609
307 345 days  1751.394148  2253.977982  0.126516
308 346 days  1757.776444  2263.923567  0.126523
309 347 days  1755.884207  2266.284941  0.126560
310 348 days  1746.399026  2262.571324  0.125502
311 349 days  1748.048607  2260.058035  0.125205
312 350 days  1764.643536  2272.244580  0.126638
313 351 days  1783.119877  2295.836265  0.127993
314 352 days  1792.085336  2309.329988  0.128340
315 353 days  1802.884778  2322.675914  0.129045
316 354 days  1806.858167  2328.249463  0.129885
317 355 days  1791.721438  2312.681016  0.128714
318 356 days  1791.212478  2309.953597  0.128516
319 357 days  1799.767680  2315.643950  0.129403
320 358 days  1809.769134  2330.864314  0.130368
321 359 days  1819.714119  2346.332356  0.131220
322 360 days  1829.183683  2361.425878  0.131605
323 361 days  1833.820766  2369.483303  0.132016
324 362 days  1829.029939  2369.958186  0.131614
325 363 days  1833.762609  2369.502036  0.131607
326 364 days  1844.281709  2372.110049  0.132707
327 365 days  1852.429514  2384.950858  0.132949
```



```
specific_dates = pd.to_datetime(['2026-02-25', '2026-02-26', '2026-02-27'])

result = forecast[
    forecast['ds'].isin(specific_dates)
][['ds', 'yhat', 'yhat_lower', 'yhat_upper']].copy()

result = result.round(2)
result.columns = ['Date', 'Predicted', 'Bear Case', 'Bull Case']
result['Day']  = result['Date'].dt.day_name()
result         = result[['Date', 'Day', 'Predicted', 'Bear Case', 'Bull Case']]
result         = result.reset_index(drop=True)

print("NIFTY 50 — Feb 25, 26, 27 Predictions")
print("=" * 65)
print(result.to_string(index=False))
```

```
NIFTY 50 — Feb 25, 26, 27 Predictions
=================================================================
      Date       Day  Predicted  Bear Case  Bull Case
2026-02-25 Wednesday   26141.99   25090.27   27138.08
2026-02-26  Thursday   26141.06   25087.25   27138.77
2026-02-27    Friday   26085.15   25095.09   27174.09
```

Retune Prophet with Better Config

```
prophet_df_log = prophet_df.copy()
prophet_df_log['y'] = np.log(prophet_df_log['y'])

print(f"Log transformed data ready")
print(f"Original last close: {prophet_df['y'].iloc[-1]:.2f}")
print(f"Log last close:      {prophet_df_log['y'].iloc[-1]:.4f}")

model_tuned = Prophet(
    changepoint_prior_scale = 0.20,
    changepoint_range = 0.95,
    n_changepoints = 40,
    yearly_seasonality = False,
    weekly_seasonality = False,
    daily_seasonality  = False,

    seasonality_prior_scale = 12,
    seasonality_mode = 'additive',
    holidays = indian_holidays_df,
    holidays_prior_scale = 10,
    interval_width = 0.95,
)
model_tuned.add_seasonality(
    name         = 'yearly',
    period       = 365.25,
    fourier_order = 15,
)

print("Training tuned Prophet on log data...")
model_tuned.fit(prophet_df_log)
print("Tuned Prophet trained!")
```

```
Log transformed data ready
Original last close: 25496.55
Log last close:      10.1463
Training tuned Prophet on log data...
Tuned Prophet trained!
```

```
!pip install xgboost -q
print("XGBoost installed!")
```

```
XGBoost installed!
```

Get Prophet Predictions on Training Data

```
future_tuned    = model_tuned.make_future_dataframe(
                    periods=365, freq='B')
forecast_tuned  = model_tuned.predict(future_tuned)
forecast_tuned['yhat']       = np.exp(forecast_tuned['yhat'])
forecast_tuned['yhat_lower'] = np.exp(forecast_tuned['yhat_lower'])
forecast_tuned['yhat_upper'] = np.exp(forecast_tuned['yhat_upper'])

print("Forecast generated and converted back from log scale!")
print(f"Forecast goes up to: {forecast_tuned['ds'].max().date()}")
merged = prophet_df.merge(
    forecast_tuned[['ds', 'yhat']],
```

```
    on='ds',
    how='inner'
)
merged['residual'] = merged['y'] - merged['yhat']

print(f"\nResidual Stats:")
print(f"Mean residual:  {merged['residual'].mean():.2f}")
print(f"Std residual:   {merged['residual'].std():.2f}")
print(f"Max residual:   {merged['residual'].max():.2f}")
print(f"Min residual:   {merged['residual'].min():.2f}")

print(f"\nSample (actual vs predicted vs residual):")
print(merged[['ds', 'y', 'yhat', 'residual']].tail(10).to_string(index=False))
```

```
Forecast generated and converted back from log scale!
Forecast goes up to: 2027-07-22

Residual Stats:
Mean residual:  6.09
Std residual:   373.73
Max residual:   1692.02
Min residual:   -2603.14

Sample (actual vs predicted vs residual):
        ds            y         yhat    residual
2026-02-13 25471.099609 25614.644964 -143.545354
2026-02-16 25682.750000 25529.486607  153.263393
2026-02-17 25725.400391 25537.667110  187.733281
2026-02-18 25819.349609 25512.416690  306.932919
2026-02-19 25454.349609 25486.836511  -32.486901
2026-02-20 25571.250000 25462.115135  109.134865
2026-02-23 25713.000000 25404.040495  308.959505
2026-02-24 25424.650391 25392.869175   31.781215
2026-02-25 25482.500000 25386.583367   95.916633
2026-02-26 25496.550781 25385.200828  111.349953
```

Engineer Features for XGBoost

```
features = merged.copy()
features = features.sort_values('ds').reset_index(drop=True)
features['day_of_week']  = features['ds'].dt.dayofweek
features['month']        = features['ds'].dt.month
features['quarter']      = features['ds'].dt.quarter
features['day_of_year']  = features['ds'].dt.dayofyear
features['week_of_year'] = features['ds'].dt.isocalendar().week.astype(int)
features['year']         = features['ds'].dt.year
features['lag_1']  = features['y'].shift(1)
features['lag_5']  = features['y'].shift(5)
features['lag_21'] = features['y'].shift(21)
features['lag_63'] = features['y'].shift(63)
features['rolling_mean_5']  = features['y'].rolling(5).mean()
features['rolling_mean_21'] = features['y'].rolling(21).mean()
features['rolling_std_5']   = features['y'].rolling(5).std()
features['rolling_std_21']  = features['y'].rolling(21).std()
features['momentum_5']  = features['y'] - features['lag_5']
features['momentum_21'] = features['y'] - features['lag_21']

features = features.dropna().reset_index(drop=True)

print(f"Features engineered!")
print(f"Total features: {len([c for c in features.columns if c not in ['ds', 'y', 'yhat', 'residual']])}")
print(f"Total rows after dropping NaN: {len(features)}")
print(f"\nFeature columns:")
feature_cols = ['day_of_week', 'month', 'quarter', 'day_of_year',
                'week_of_year', 'year', 'lag_1', 'lag_5', 'lag_21',
                'lag_63', 'rolling_mean_5', 'rolling_mean_21',
                'rolling_std_5', 'rolling_std_21',
                'momentum_5', 'momentum_21']
print(feature_cols)
```

```
Features engineered!
Total features: 16
Total rows after dropping NaN: 3905

Feature columns:
['day_of_week', 'month', 'quarter', 'day_of_year', 'week_of_year', 'year', 'lag_1', 'lag_5', 'lag_21', 'lag_63',
```

Train XGBoost on Residuals

```
from xgboost import XGBRegressor
from sklearn.model_selection import TimeSeriesSplit
```

```
from sklearn.metrics import mean_absolute_error
import warnings
warnings.filterwarnings('ignore')
X = features[feature_cols]
y = features['residual']
split_idx   = int(len(X) * 0.80)
X_train     = X.iloc[:split_idx]
X_test      = X.iloc[split_idx:]
y_train     = y.iloc[:split_idx]
y_test      = y.iloc[split_idx:]

print(f"Train size: {len(X_train)} days")
print(f"Test size:  {len(X_test)} days")
print(f"Train up to: {features['ds'].iloc[split_idx-1].date()}")
print(f"Test from:   {features['ds'].iloc[split_idx].date()}")

xgb_model = XGBRegressor(
    n_estimators      = 500,
    learning_rate     = 0.05,
    max_depth         = 5,
    subsample         = 0.8,
    colsample_bytree  = 0.8,
    random_state      = 42,
    early_stopping_rounds = 50,
)

print("Training XGBoost on Prophet residuals...")
xgb_model.fit(
    X_train, y_train,
    eval_set        = [(X_test, y_test)],
    verbose         = False
)
print("XGBoost trained!")
import pandas as pd
importance = pd.DataFrame({
    'Feature'   : feature_cols,
    'Importance': xgb_model.feature_importances_
}).sort_values('Importance', ascending=False)

print("\nWhich features matter most to XGBoost:")
print(importance.to_string(index=False))
```

```
Train size: 3124 days
Test size:  781 days
Train up to: 2022-12-28
Test from:   2022-12-29
Training XGBoost on Prophet residuals...
XGBoost trained!

Which features matter most to XGBoost:
        Feature  Importance
    momentum_21    0.223482
 rolling_std_21    0.100975
 rolling_mean_5    0.091226
rolling_mean_21    0.089237
           year    0.071595
         lag_63    0.057645
   week_of_year    0.056430
          lag_1    0.056289
        quarter    0.055317
          lag_5    0.043842
         lag_21    0.043472
    day_of_year    0.034226
     momentum_5    0.032481
          month    0.027171
  rolling_std_5    0.014584
    day_of_week    0.002027
```

Combine Prophet + XGBoost Predictions

```
test_data = features.iloc[split_idx:].copy()
prophet_pred = test_data['yhat'].values
xgb_correction = xgb_model.predict(X_test)
combined_pred = prophet_pred + xgb_correction
actual = test_data['y'].values

mae_prophet  = mean_absolute_error(actual, prophet_pred)
mae_combined = mean_absolute_error(actual, combined_pred)

mape_prophet  = np.mean(np.abs((actual - prophet_pred)  / actual)) * 100
mape_combined = np.mean(np.abs((actual - combined_pred) / actual)) * 100

improvement = ((mape_prophet - mape_combined) / mape_prophet) * 100
```

```
print("RESULTS — Prophet vs Prophet + XGBoost")
print("=" * 55)
print(f"{'Metric':<25} {'Prophet':>12} {'Combined':>12}")
print("-" * 55)
print(f"{'MAE (points)':<25} {mae_prophet:>12.2f} {mae_combined:>12.2f}")
print(f"{'MAPE (%)':<25} {mape_prophet:>12.2f} {mape_combined:>12.2f}")
print(f"{'Accuracy (%)':<25} {100-mape_prophet:>12.2f} {100-mape_combined:>12.2f}")
print("=" * 55)
print(f"\nImprovement: {improvement:.1f}% reduction in error")
print(f"Prophet alone:   {100-mape_prophet:.1f}% accurate")
print(f"Prophet+XGBoost: {100-mape_combined:.1f}% accurate")
```

```
RESULTS — Prophet vs Prophet + XGBoost
=======================================================
Metric                        Prophet     Combined
-------------------------------------------------------
MAE (points)                   368.63       353.97
MAPE (%)                         1.65         1.59
Accuracy (%)                    98.35        98.41
=======================================================

Improvement: 3.6% reduction in error
Prophet alone:   98.4% accurate
Prophet+XGBoost: 98.4% accurate
```

```
specific_dates = pd.to_datetime(['2026-02-25', '2026-02-26', '2026-02-27'])
result_tuned = forecast_tuned[
    forecast_tuned['ds'].isin(specific_dates)
][['ds', 'yhat', 'yhat_lower', 'yhat_upper']].copy().round(2)

result_tuned.columns = ['Date', 'Predicted', 'Bear Case', 'Bull Case']
result_tuned['Day']  = result_tuned['Date'].dt.day_name()
result_tuned         = result_tuned[
    ['Date', 'Day', 'Predicted', 'Bear Case', 'Bull Case']
].reset_index(drop=True)

print("NIFTY 50 — Tuned Prophet Predictions")
print("=" * 65)
print(result_tuned.to_string(index=False))

import yfinance as yf

actual_df = yf.download("^NSEI",
                        start='2026-02-25',
                        end='2026-02-28')
actual_df.columns = actual_df.columns.get_level_values(0)
actual_df.reset_index(inplace=True)
actual_df['Date'] = pd.to_datetime(actual_df['Date'])
actual_df = actual_df[['Date', 'Close']].round(2)

comparison = result_tuned.merge(actual_df, on='Date', how='left')
comparison.rename(columns={'Close': 'Actual'}, inplace=True)

print("\nTuned Prophet — Predicted vs Actual")
print("=" * 80)

errors = []
for _, row in comparison.iterrows():
    if pd.notna(row['Actual']):
        error_pts  = row['Actual'] - row['Predicted']
        error_pct  = (error_pts / row['Actual']) * 100
        in_band    = '✅' if row['Bear Case'] <= row['Actual'] <= row['Bull Case'] else '❌'
        arrow      = "📈" if error_pts > 0 else "📉"
        errors.append(abs(error_pct))

        print(f"{arrow} {row['Day']:10} {str(row['Date'].date())}  |  "
              f"Predicted: {row['Predicted']:>10,.2f}  |  "
              f"Actual: {row['Actual']:>10,.2f}  |  "
              f"Error: {error_pts:>+8.2f} ({error_pct:>+.2f}%)  |  "
              f"In Band: {in_band}")
    else:
        print(f"{row['Day']:10} {str(row['Date'].date())}  |  "
              f"Predicted: {row['Predicted']:>10,.2f}  |  "
              f"Actual: Not available yet")
if errors:
    avg_error = sum(errors) / len(errors)
    print(f"\nAverage Error:       {avg_error:.2f}%")
    print(f"Old model error:     ~2.90%")
    print(f"Improvement:         {((2.90 - avg_error) / 2.90 * 100):.1f}% better than old model")
    print(f"Expected MAPE:       1.65%")
    print(f"Verdict: {'Matches expected!' if avg_error < 2.0 else 'Check results'}")
```

```
[*********************100%***********************]  1 of 1 completedNIFTY 50 — Tuned Prophet Predictions
==================================================================
      Date       Day  Predicted  Bear Case  Bull Case
2026-02-25 Wednesday   25386.58   23925.03   27077.32
2026-02-26  Thursday   25385.20   23626.21   27113.03
2026-02-27    Friday   25388.41   23929.17   27095.20

Tuned Prophet — Predicted vs Actual
================================================================================
📈 Wednesday  2026-02-25  |  Predicted:  25,386.58  |  Actual:  25,482.50  |  Error:   +95.92 (+0.38%)  |  In Bar
📈 Thursday   2026-02-26  |  Predicted:  25,385.20  |  Actual:  25,496.55  |  Error:  +111.35 (+0.44%)  |  In Bar
📉 Friday     2026-02-27  |  Predicted:  25,388.41  |  Actual:  25,178.65  |  Error:  -209.76 (-0.83%)  |  In Bar

Average Error:       0.55%
Old model error:     ~2.90%
Improvement:         81.1% better than old model
Expected MAPE:       1.65%
Verdict: Matches expected!
```

```
!pip install tensorflow -q

import numpy as np
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error
import warnings
warnings.filterwarnings('ignore')

print("All LSTM libraries imported!")
```

```
All LSTM libraries imported!
```

Prepare Data for LSTM

```
SEQUENCE_LENGTH = 60
prices = prophet_df['y'].values.reshape(-1, 1)
scaler = MinMaxScaler(feature_range=(0, 1))
prices_scaled = scaler.fit_transform(prices)

print(f"Data scaled to 0-1 range")
print(f"Original range: {prices.min():.2f} to {prices.max():.2f}")
print(f"Scaled range:   {prices_scaled.min():.4f} to {prices_scaled.max():.4f}")
print(f"Total data points: {len(prices_scaled)}")


def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(seq_length, len(data)):
        X.append(data[i-seq_length:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)

X_lstm, y_lstm = create_sequences(prices_scaled, SEQUENCE_LENGTH)
X_lstm = X_lstm.reshape(X_lstm.shape[0], X_lstm.shape[1], 1)

print(f"\nSequences created!")
print(f"X shape: {X_lstm.shape}")
print(f"   → {X_lstm.shape[0]} sequences")
print(f"   → each sequence = {X_lstm.shape[1]} days")
print(f"   → {X_lstm.shape[2]} feature per day (price)")
print(f"y shape: {y_lstm.shape}")
print(f"   → {y_lstm.shape[0]} targets (next day prices)")
```

```
Data scaled to 0-1 range
Original range: 4544.20 to 26328.55
Scaled range:   0.0000 to 1.0000
Total data points: 3968

Sequences created!
X shape: (3908, 60, 1)
   → 3908 sequences
   → each sequence = 60 days
   → 1 feature per day (price)
y shape: (3908,)
   → 3908 targets (next day prices)
```

Split Data & Build LSTM Model

```
split_idx    = int(len(X_lstm) * 0.80)

X_train_lstm = X_lstm[:split_idx]
X_test_lstm  = X_lstm[split_idx:]
y_train_lstm = y_lstm[:split_idx]
y_test_lstm  = y_lstm[split_idx:]

print(f"Train size: {len(X_train_lstm)} sequences")
print(f"Test size:  {len(X_test_lstm)} sequences")

model_lstm = Sequential([

    LSTM(units       = 128,
         return_sequences = True,
         input_shape  = (SEQUENCE_LENGTH, 1)),
    Dropout(0.2),
    LSTM(units       = 64,
         return_sequences = False),
    Dropout(0.2),
    Dense(units = 32,
          activation = 'relu'),
    Dense(units = 1)
])

model_lstm.compile(
    optimizer = 'adam',
    loss      = 'mean_squared_error'
)

model_lstm.summary()
print("\nLSTM model built!")
```

```
Train size: 3126 sequences
Test size:  782 sequences
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 128)	66,560
dropout (Dropout)	(None, 60, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 118,081 (461.25 KB)
Trainable params: 118,081 (461.25 KB)
Non-trainable params: 0 (0.00 B)

```
LSTM model built!
```

Train LSTM

```
early_stop = EarlyStopping(
    monitor   = 'val_loss',
    patience  = 15,
    restore_best_weights = True
)

print("Training LSTM (takes 3-5 mins)...")
print("Watch val_loss — should decrease over epochs")
print("If val_loss starts increasing = overfitting")
print()

history = model_lstm.fit(
    X_train_lstm, y_train_lstm,
    epochs          = 100,
    batch_size      = 32,
    validation_split = 0.10,
    callbacks       = [early_stop],
    verbose         = 1
)

print("\nLSTM training complete!")
print(f"Trained for {len(history.history['loss'])} epochs")
print(f"Final train loss:      {history.history['loss'][-1]:.6f}")
print(f"Final validation loss: {history.history['val_loss'][-1]:.6f}")
```

```
88/88 ━━━━━━━━━━━━━━━━━━━━ 19s 109ms/step - loss: 1.3462e-04 - val_loss: 2.7493e-04
Epoch 34/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 10s 108ms/step - loss: 1.3675e-04 - val_loss: 0.0010
Epoch 35/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 123ms/step - loss: 1.5931e-04 - val_loss: 1.7328e-04
Epoch 36/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 120ms/step - loss: 1.2199e-04 - val_loss: 1.2965e-04
Epoch 37/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 123ms/step - loss: 1.2334e-04 - val_loss: 2.8827e-04
Epoch 38/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 10s 111ms/step - loss: 1.0727e-04 - val_loss: 1.4481e-04
Epoch 39/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 10s 112ms/step - loss: 1.0723e-04 - val_loss: 1.3961e-04
Epoch 40/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 120ms/step - loss: 1.1602e-04 - val_loss: 1.2988e-04
Epoch 41/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 125ms/step - loss: 1.2116e-04 - val_loss: 1.7058e-04
Epoch 42/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 123ms/step - loss: 1.2984e-04 - val_loss: 4.5602e-04
Epoch 43/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 10s 109ms/step - loss: 1.1566e-04 - val_loss: 1.6527e-04
Epoch 44/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 10s 118ms/step - loss: 1.3222e-04 - val_loss: 1.2555e-04
Epoch 45/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 119ms/step - loss: 1.3272e-04 - val_loss: 0.0011
Epoch 46/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 10s 118ms/step - loss: 1.1318e-04 - val_loss: 1.3147e-04
Epoch 47/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 10s 119ms/step - loss: 1.0480e-04 - val_loss: 1.4843e-04
Epoch 48/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 9s 104ms/step - loss: 1.2247e-04 - val_loss: 4.1975e-04
Epoch 49/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 10s 119ms/step - loss: 9.7176e-05 - val_loss: 0.0010
Epoch 50/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 123ms/step - loss: 1.0709e-04 - val_loss: 0.0012
Epoch 51/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 20s 119ms/step - loss: 1.1263e-04 - val_loss: 8.3778e-04
Epoch 52/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 9s 105ms/step - loss: 1.0528e-04 - val_loss: 6.6533e-04
Epoch 53/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 119ms/step - loss: 1.0100e-04 - val_loss: 5.4326e-04
Epoch 54/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 122ms/step - loss: 1.2805e-04 - val_loss: 6.8692e-04
Epoch 55/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 123ms/step - loss: 1.1011e-04 - val_loss: 0.0012
Epoch 56/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 19s 104ms/step - loss: 1.0312e-04 - val_loss: 0.0020
Epoch 57/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 124ms/step - loss: 1.0219e-04 - val_loss: 0.0024
Epoch 58/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 11s 119ms/step - loss: 1.0144e-04 - val_loss: 0.0012
Epoch 59/100
88/88 ━━━━━━━━━━━━━━━━━━━━ 20s 120ms/step - loss: 8.9250e-05 - val_loss: 6.9983e-04

LSTM training complete!
Trained for 59 epochs
Final train loss:      0.000090
Final validation loss: 0.000700
```

Compare All 3 Models

```
test_dates = prophet_df['ds'].iloc[
    split_idx + SEQUENCE_LENGTH:
].reset_index(drop=True)

test_actual = prophet_df['y'].iloc[
    split_idx + SEQUENCE_LENGTH:
].values
lstm_pred_scaled = model_lstm.predict(X_test_lstm, verbose=0)
lstm_pred = scaler.inverse_transform(lstm_pred_scaled).flatten()
prophet_test = forecast_tuned[
    forecast_tuned['ds'].isin(test_dates)
]['yhat'].values
min_len      = min(len(test_actual), len(lstm_pred),
                   len(prophet_test))
test_actual  = test_actual[:min_len]
lstm_pred    = lstm_pred[:min_len]
prophet_test = prophet_test[:min_len]
features_test = features.iloc[split_idx:split_idx+min_len].copy()
if len(features_test) == min_len:
    xgb_correction_test = xgb_model.predict(
        features_test[feature_cols]
    )
    prophet_xgb_pred = prophet_test + xgb_correction_test
else:
    prophet_xgb_pred = prophet_test
```

```
w_prophet = 0.40
w_xgb     = 0.35
w_lstm    = 0.25

ensemble_pred = (
    w_prophet * prophet_test  +
    w_xgb     * prophet_xgb_pred +
    w_lstm    * lstm_pred
)
def mape(actual, predicted):
    return np.mean(np.abs((actual - predicted) / actual)) * 100

mape_prophet  = mape(test_actual, prophet_test)
mape_xgb      = mape(test_actual, prophet_xgb_pred)
mape_lstm     = mape(test_actual, lstm_pred)
mape_ensemble = mape(test_actual, ensemble_pred)

print("FINAL RESULTS — All Models Compared")
print("=" * 60)
print(f"{'Model':<30} {'MAPE':>8} {'Accuracy':>10}")
print("-" * 60)
print(f"{'Prophet tuned alone':<30} {mape_prophet:>7.2f}% {100-mape_prophet:>9.2f}%")
print(f"{'Prophet tuned + XGBoost':<30} {mape_xgb:>7.2f}% {100-mape_xgb:>9.2f}%")
print(f"{'LSTM alone':<30} {mape_lstm:>7.2f}% {100-mape_lstm:>9.2f}%")
print(f"{'Prophet + XGBoost + LSTM':<30} {mape_ensemble:>7.2f}% {100-mape_ensemble:>9.2f}%")
print("=" * 60)

best_mape  = min(mape_prophet, mape_xgb, mape_lstm, mape_ensemble)
best_model = ['Prophet tuned', 'Prophet+XGBoost',
              'LSTM', 'Full Ensemble'][
    [mape_prophet, mape_xgb, mape_lstm, mape_ensemble].index(best_mape)
]
print(f"\n🏆 Best model: {best_model} at {100-best_mape:.2f}% accuracy")
```

```
FINAL RESULTS — All Models Compared
============================================================
Model                              MAPE   Accuracy
------------------------------------------------------------
Prophet tuned alone               1.65%     98.35%
Prophet tuned + XGBoost           1.65%     98.35%
LSTM alone                        1.50%     98.50%
Prophet + XGBoost + LSTM          1.32%     98.68%
============================================================

🏆 Best model: Full Ensemble at 98.68% accuracy
```

Previous days prediction

```
specific_dates = pd.to_datetime(['2026-02-25', '2026-02-26', '2026-02-27'])
prophet_specific = forecast_tuned[
    forecast_tuned['ds'].isin(specific_dates)
][['ds', 'yhat']].copy().reset_index(drop=True)

lstm_preds = []
for target_date in specific_dates:
    idx = prophet_df[prophet_df['ds'] == target_date].index
    if len(idx) > 0:
        idx = idx[0]
        seq = prophet_df['y'].iloc[idx-60:idx].values.reshape(-1, 1)
        seq_scaled  = scaler.transform(seq)
        seq_input   = seq_scaled.reshape(1, 60, 1)
        pred_scaled = model_lstm.predict(seq_input, verbose=0)
        pred_price  = scaler.inverse_transform(pred_scaled)[0][0]
        lstm_preds.append(pred_price)
    else:
        lstm_preds.append(None)

xgb_corrections = []
for target_date in specific_dates:
    row_data = features[features['ds'] == target_date]
    if len(row_data) > 0:
        corr = xgb_model.predict(row_data[feature_cols])[0]
    else:
        corr = 0.0
    xgb_corrections.append(corr)

actual_df = yf.download("^NSEI",
                        start='2026-02-25',
                        end='2026-02-28')
actual_df.columns = actual_df.columns.get_level_values(0)
actual_df.reset_index(inplace=True)
actual_df['Date'] = pd.to_datetime(actual_df['Date'])
```

```
actual_df = actual_df[['Date', 'Close']].round(2)
actual_dict = dict(zip(actual_df['Date'], actual_df['Close']))

print("Feb 25, 26, 27 — All Models vs Actual")
print("=" * 95)
print(f"{'Day':<10} {'Date':<12} {'Prophet':>10} {'LSTM':>10} "
      f"{'Ensemble':>10} {'Actual':>10} {'Error%':>8} {'InBand':>8}")
print("-" * 95)

errors = []
for i, (_, row) in enumerate(prophet_specific.iterrows()):
    date_val    = row['ds']
    prophet_p   = row['yhat']
    lstm_p      = lstm_preds[i] if lstm_preds[i] else prophet_p
    xgb_corr    = xgb_corrections[i]
    prophet_xgb = prophet_p + xgb_corr

    final = (0.40 * prophet_p +
             0.35 * prophet_xgb +
             0.25 * lstm_p)

    actual    = actual_dict.get(date_val)
    bear_case = forecast_tuned[
        forecast_tuned['ds'] == date_val
    ]['yhat_lower'].values[0]
    bull_case = forecast_tuned[
        forecast_tuned['ds'] == date_val
    ]['yhat_upper'].values[0]

    if actual:
        error_pct = abs((actual - final) / actual) * 100
        in_band   = '✅' if bear_case <= actual <= bull_case else '❌'
        arrow     = "📈" if final > actual else "📉"
        errors.append(error_pct)

        print(f"{arrow} {date_val.day_name():<10} "
              f"{str(date_val.date()):<12} "
              f"{prophet_p:>10,.2f} "
              f"{lstm_p:>10,.2f} "
              f"{final:>10,.2f} "
              f"{actual:>10,.2f} "
              f"{error_pct:>7.2f}% "
              f"{in_band:>8}")
    else:
        print(f"⏳ {date_val.day_name():<10} "
              f"{str(date_val.date()):<12} "
              f"{prophet_p:>10,.2f} "
              f"{lstm_p:>10,.2f} "
              f"{final:>10,.2f} "
              f"{'N/A':>10} "
              f"{'N/A':>8} "
              f"{'⏳':>8}")

print("=" * 95)
if errors:
    avg_error = sum(errors) / len(errors)
    print(f"\nAverage Error:         {avg_error:.2f}%")
    print(f"Old basic model:       ~2.90%")
    print(f"Tuned Prophet:         ~1.65%")
    print(f"This ensemble:          {avg_error:.2f}%")
    print(f"Improvement vs old:    {((2.90-avg_error)/2.90*100):.1f}% better")
    print(f"Improvement vs tuned:  {((1.65-avg_error)/1.65*100):.1f}% better")
    print(f"\nBest model: {'Ensemble!' if avg_error < 1.65 else 'Tuned Prophet'}")
```

```
[*********************100%***********************]  1 of 1 completedFeb 25, 26, 27 — All Models vs Actual
===============================================================================================
Day         Date            Prophet       LSTM   Ensemble     Actual   Error%   InBand
-----------------------------------------------------------------------------------------------
📈 Wednesday  2026-02-25    25,386.58  26,238.33  25,651.54  25,482.50    0.66%        ✅
📈 Thursday   2026-02-26    25,385.20  26,196.66  25,629.21  25,496.55    0.52%        ✅
📈 Friday     2026-02-27    25,388.41  25,388.41  25,388.41  25,178.65    0.83%        ✅
===============================================================================================

Average Error:         0.67%
Old basic model:       ~2.90%
Tuned Prophet:         ~1.65%
This ensemble:          0.67%
Improvement vs old:    76.8% better
Improvement vs tuned:  59.3% better

Best model: Ensemble!
```

```
!pip install mistralai requests -q
print("Libraries installed!")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 509.3/509.3 kB 30.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 160.3/160.3 kB 13.0 MB/s eta 0:00:00
Libraries installed!
```

```
from mistralai import Mistral

MISTRAL_API_KEY = "RcV8PCSdIxEhOiJdRvf7tFRuzcRzCv98"
MISTRAL_MODEL   = "mistral-small-latest"

client = Mistral(api_key=MISTRAL_API_KEY)
print("Mistral client ready!")
```

```
Mistral client ready!
```

Fetch Live News Function

```
import requests
from datetime import date, timedelta

def fetch_nifty_news():
    """
    Fetch latest NIFTY/Indian market news
    using NewsAPI free tier
    Sign up free at: newsapi.org
    OR we scrape headlines directly
    """
    try:
        headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
        }

        url = "https://economictimes.indiatimes.com/markets/stocks/news"
        response = requests.get(url, headers=headers, timeout=10)

        if response.status_code == 200:
            from bs4 import BeautifulSoup
            soup = BeautifulSoup(response.content, 'html.parser')

            headlines = []
            for tag in soup.find_all(['h3', 'h4'], limit=15):
                text = tag.get_text(strip=True)
                if len(text) > 20:
                    headlines.append(text)

            return headlines[:10]
        else:
            return ["Could not fetch live news"]

    except Exception as e:
        print(f"News fetch failed: {e}")
        return ["News unavailable — using data only"]


def fetch_recent_nifty_context():
    """
    Build context from recent NIFTY data we already have
    Last 30 days movement, trend, volatility
    """
    recent = prophet_df.tail(30).copy()

    last_price    = recent['y'].iloc[-1]
    price_30d_ago = recent['y'].iloc[0]
    price_7d_ago  = recent['y'].iloc[-5]

    change_30d    = ((last_price - price_30d_ago) / price_30d_ago) * 100
    change_7d     = ((last_price - price_7d_ago)  / price_7d_ago)  * 100
    volatility    = recent['y'].pct_change().std() * 100

    recent_returns = recent['y'].pct_change().tail(7).values * 100
    recent_returns = [f"{r:+.2f}%" for r in recent_returns if not np.isnan(r)]

    context = f"""
    NIFTY 50 Recent Data Summary:
    ────────────────────────────
    Current Price:        {last_price:,.2f}
    7 days ago:           {price_7d_ago:,.2f}
    30 days ago:          {price_30d_ago:,.2f}
```

```
    7-day change:         {change_7d:+.2f}%
    30-day change:        {change_30d:+.2f}%
    Recent volatility:    {volatility:.2f}% daily std

    Last 7 daily returns: {', '.join(recent_returns)}

    Trend direction:      {'📈 Upward' if change_7d > 0 else '📉 Downward'}
    Market condition:     {'High volatility' if volatility > 1 else 'Low volatility'}
    """

    return context

print("📰 Fetching live news...")
news = fetch_nifty_news()
print(f"Got {len(news)} headlines!")
for i, h in enumerate(news, 1):
    print(f"  {i}. {h}")

print("\nBuilding NIFTY context...")
context = fetch_recent_nifty_context()
print(context)
```

```
📰 Fetching live news...
Got 10 headlines!
  1. Adani Ports shares plunge 6% as escalating Iran-Israel war sparks concerns around Haifa port
  2. Pakistan's KSE-30 index crashes 10%, biggest drop on record as Mideast tensions escalate
  3. FIIs doubled their stake in 6 stocks; 3 surge over 15% just 2 months
  4. Shanghai stocks close at 10-year high as Iran conflict boosts energy and gold shares; HK drops
  5. Stock Market Holi Holiday 2026: Will NSE & BSE be closed on March 3 or 4?
  6. European shares touch two-week lows on Middle East conflict
  7. LT Foods, KRBL & other rice stocks fall up to 11% as Israel-Iran war flares up. Here's why
  8. AI Tool Sudarshan: SEBI Chief shares how unethical finfluencer are being weeded out
  9. Best places to hide in war: 20 stocks analysts say to bet on amid US-Iran conflict
  10. Jefferies calls Israel-Iran conflict a buying opportunity; warns Hormuz, crude shock risks for India

Building NIFTY context...

    NIFTY 50 Recent Data Summary:
    ──────────────────────────────
    Current Price:        25,496.55
    7 days ago:           25,571.25
    30 days ago:          25,665.60

    7-day change:         -0.29%
    30-day change:        -0.66%
    Recent volatility:    0.83% daily std

    Last 7 daily returns: +0.37%, -1.41%, +0.46%, +0.55%, -1.12%, +0.23%, +0.06%

    Trend direction:      📉 Downward
    Market condition:     Low volatility
```

Mistral Adjustment Function

```
def get_mistral_adjustment(prophet_prediction, target_date):
    """
    Ask Mistral to analyze context + news
    and suggest adjustment to Prophet's prediction
    """
    print("Fetching latest news...")
    news_headlines = fetch_nifty_news()
    news_text = "\n".join([f"- {h}" for h in news_headlines])

    nifty_context = fetch_recent_nifty_context()
    prompt = f"""
You are an expert Indian stock market analyst specializing in NIFTY 50.

Your job is to analyze the provided context and suggest a percentage adjustment
to a quantitative model's prediction for NIFTY 50.

═══════════════════════════════════════════════
QUANTITATIVE MODEL PREDICTION
═══════════════════════════════════════════════
Target Date:       {target_date}
Prophet Predicted: {prophet_prediction:,.2f}

═══════════════════════════════════════════════
RECENT NIFTY 50 DATA
═══════════════════════════════════════════════
{nifty_context}

═══════════════════════════════════════════════
LATEST MARKET NEWS HEADLINES
```

```
═══════════════════════════════════════════════
{news_text}

═══════════════════════════════════════════════
YOUR TASK
═══════════════════════════════════════════════
Based on:
1. The recent price trend and momentum
2. Current market news and sentiment
3. Any upcoming events or risks visible in headlines

Suggest a percentage adjustment to the prediction.

Rules:
- Be conservative — markets are hard to predict
- Only suggest adjustment if there is CLEAR evidence
- Small adjustments are better than large ones
- Typical range: -2% to +2%
- If no clear signal → suggest 0% adjustment

Respond in EXACTLY this JSON format and nothing else:
{{
  "adjustment_pct": 0.5,
  "direction": "up",
  "confidence": "medium",
  "reasoning": "Brief reason in one sentence",
  "key_factor": "The single most important factor"
}}
"""

    print("Asking Mistral for market analysis...")

    response = client.chat.complete(
        model    = MISTRAL_MODEL,
        messages = [
            {
                "role"   : "system",
                "content": "You are an expert quantitative analyst. Always respond with valid JSON only."
            },
            {
                "role"   : "user",
                "content": prompt
            }
        ],
        temperature = 0.1,
    )


    import json
    response_text = response.choices[0].message.content.strip()
    if "```json" in response_text:
        response_text = response_text.split("```json")[1].split("```")[0].strip()
    elif "```" in response_text:
        response_text = response_text.split("```")[1].split("```")[0].strip()

    result = json.loads(response_text)
    return result

test_prediction = forecast_tuned[
    forecast_tuned['ds'] == '2026-02-27'
]['yhat'].values[0]

print(f"Prophet prediction: {test_prediction:,.2f}")
print()

mistral_result = get_mistral_adjustment(test_prediction, '2026-02-27')

print(f"\nMistral Analysis:")
print(f"   Adjustment:  {mistral_result['adjustment_pct']:+.2f}%")
print(f"   Direction:   {mistral_result['direction']}")
print(f"   Confidence:  {mistral_result['confidence']}")
print(f"   Reasoning:   {mistral_result['reasoning']}")
print(f"   Key Factor:  {mistral_result['key_factor']}")
```

```
Prophet prediction: 25,388.41

Fetching latest news...
Asking Mistral for market analysis...

Mistral Analysis:
   Adjustment:  +1.50%
   Direction:   down
   Confidence:  medium
```